

SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 2- DEBUGGING & OPTIMISATION TASK

Course Code:	CSA1205 23	Course Title:	Computer Architecture for Multicore Systems
CO Assessed:	CO2	Assessment:	ASSESSMENT TOOL3- DEBUGGING & OPTIMISATION TASK
Weightage:	20%	Total Marks:	25
CO2 Statement:	Apply integer and floating-point arithmetic algorithms including Booth's multiplication, restoring/non-restoring division, and IEEE 754 standard operations to solve fixed-point and floating-point computational problems. (BL3)		
Student Name:	M. Virendra	Reg.No.:	152471046
Department:	CSE & BS	Date:	

ANNEXURE A

1. A program performing signed integer addition using 2's complement produces incorrect results when adding large values (e.g., +120 and +50 in 8-bit representation). Debug the issue by analyzing the binary computation and identifying whether overflow has occurred. Propose a solution to correctly detect and handle overflow in such cases.
2. A processor using a ripple carry adder shows significant delay in executing arithmetic operations in a real-time system. Debug the performance bottleneck by examining how carry propagates through each bit. Suggest an optimized solution using a carry look-ahead adder, and explain how it reduces delay.
3. A multiplication unit implementing Booth's algorithm gives incorrect results for certain negative operands. Debug the step-by-step execution of the algorithm, focusing on bit-pair checking, arithmetic shifts, and sign extension. Identify the possible source of error and suggest corrections to ensure accurate signed multiplication.
4. An embedded system using the restoring division algorithm experiences inefficiency due to repeated restoration steps, leading to increased execution time. Debug the algorithm's workflow and identify where unnecessary operations occur. Propose an optimized approach using the non-restoring division method, explaining how it improves performance.
5. A scientific application using IEEE 754 single precision floating-point representation produces inaccurate results when adding very small and very large numbers. Debug the issue by analyzing exponent alignment, normalization, and rounding errors. Suggest optimization techniques, such as using double precision or algorithmic adjustments, to improve accuracy.

Code of Conduct:

I M. Nirudha (192421040) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

M. Nirudha
Signature of the student:

MARKS ALLOCATION

	Identification of Errors / Bugs (20%)	Debugging Approach & Analysis (20%)	Correctness of Debugged Solution (25%)	Optimization Techniques Applied (20%)	Testing and Documentation (15%)	
Q1	4	3	4	4	2	17
Q2	3	4	5	3	2	17
Q3	4	4	4	3	1	16
Q4	3	3	3	4	2	15
Q5	4	4	4	4	2	18

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Identification of Errors / Bugs	20%	Accurately identifies all errors and issues with complete understanding of the problem	Identifies most errors with minor omissions	Identifies limited errors; misses key issues	Unable to identify errors or misunderstands the problem
Debugging Approach & Analysis	20%	Uses effective debugging techniques with clear analysis and root cause identification	Applies suitable debugging methods with minor gaps	Uses basic debugging methods with limited analysis	No systematic debugging approach followed
Correctness of Debugged Solution	25%	Provides completely corrected solution with accurate functionality and expected output	Provides working solution with minor errors	Partially correct solution with limited functionality	Incorrect solution or fails to resolve errors
Optimization Techniques Applied	20%	Applies efficient optimization methods to improve performance, memory usage and quality	Performs optimization with noticeable improvements	Limited optimization with minimal improvements	No optimization applied or inefficient implementation
Testing and Documentation	15%	Performs complete testing with proper validation and clear documentation	Provides adequate testing and documentation with minor issues	Limited testing and incomplete documentation	No proper testing or documentation provided

SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 3- DEBUGGING & OPTIMISATION TASK

Course Code:	CSA1205	Course Title:	Computer Architecture for Multicore Systems
CO Assessed:	CO2	Assessment:	ASSESSMENT TOOL3- DEBUGGING & OPTIMISATION TASK
Weightage:	20%	Total Marks:	25
CO2 Statement:	<i>Apply integer and floating-point arithmetic algorithms including Booth's multiplication, restoring/non-restoring division, and IEEE 754 standard operations to solve fixed-point and floating-point computational problems. (BL3)</i>		
Student Name:		Reg.No.:	
Department:		Date:	

ANNEXURE A

1. A program performing signed integer addition using 2's complement produces incorrect results when adding large values (e.g., +120 and +50 in 8-bit representation). Debug the issue by analyzing the binary computation and identifying whether overflow has occurred. Propose a solution to correctly detect and handle overflow in such cases.

Debugging Signed Integer Addition Overflow

Problem

A program performing signed integer addition using 8-bit 2's complement representation produces an incorrect result when adding +120 and +50.

Analysis

Convert the numbers into 8-bit binary:

➤ **+120 = 01111000**

➤ **+50 = 00110010**

Perform binary addition:

01111000

+00110010

10101010

The obtained result is 10101010.

In 8-bit 2's complement, 10101010 = -86.

Expected decimal result:

$$120+50=170$$

Error / Cause

The valid range of an 8-bit signed integer is:

$$-128 \text{ to } +127$$

The value 170 cannot be represented in 8 bits.

Since both operands are positive but the result becomes negative, signed overflow has occurred.

Corrected / Optimized Solution

Overflow should be detected whenever:

- The sign bits of both operands are the same.
- The sign bit of the result is different.

Overflow detection logic:

```
if ((A7 == B7) && (R7 != A7))
```

```
overflow = 1;
```

Where:

- A_7 = sign bit of first operand
- B_7 = sign bit of second operand
- R_7 = sign bit of result

Another method:

$$\text{Overflow} = C_7 \oplus C_8$$

Where:

- C_7 = carry into the MSB
- C_8 = carry out of the MSB

Testing / Validation

Input: A = 01111000 (+120)

B = 00110010 (+50)

Binary Result: 10101010

Overflow Flag: 1 (Set)

Result

The binary addition is performed correctly, but the 8-bit result is invalid because signed overflow occurs. Proper overflow detection using sign-bit comparison or carry-bit comparison allows the processor to identify and handle the error correctly, ensuring reliable arithmetic operations.

2. A processor using a ripple carry adder shows significant delay in executing arithmetic operations in a real-time system. Debug the performance bottleneck by examining how

carry propagates through each bit. Suggest an optimized solution using a carry look-ahead adder, and explain how it reduces delay.

Debugging Ripple Carry Adder Delay

Problem

A processor using a ripple carry adder (RCA) experiences significant delay while executing arithmetic operations in a real-time system.

Analysis

In a ripple carry adder, the carry generated by each bit must propagate to the next bit before the next sum can be calculated.

For an 8-bit addition:

$$C_0 \rightarrow C_1 \rightarrow C_2 \rightarrow C_3 \rightarrow C_4 \rightarrow C_5 \rightarrow C_6 \rightarrow C_7$$

Assume each full adder requires 2 ns to generate its carry.

$$\text{Total delay} = 8 \times 2 = 16 \text{ ns}$$

The final sum bit cannot be produced until the carry reaches the most significant bit.

Error / Cause

The performance bottleneck is sequential carry propagation. Each stage waits for the previous stage, creating a long critical path and increasing the execution time of arithmetic operations.

Corrected / Optimized Solution

Replace the ripple carry adder with a Carry Look-Ahead Adder (CLA).

For each bit:

$$G_i = A_i B_i$$

$$P_i = A_i \oplus B_i$$

Where:

- $G_i G_{i-1} G_i$ = Generate signal
- $P_i P_{i-1} P_i$ = Propagate signal

Carries are computed in parallel:

$$C_1 = G_0 + P_0 C_0$$

$$C_2 = G_1 + P_1 G_0 + P_1 P_0 C_0$$

Testing / Validation

Input:

$$A = 10110110$$

$$B = 01011101$$

RCA Delay: 16 ns

CLA Delay: 4–5 ns

Observed: Correct sum produced with reduced delay

Result

The Carry Look-Ahead Adder reduces the delay from approximately 16 ns to about 4–5 ns for an 8-bit operation. By generating carries in parallel, it removes the carry propagation bottleneck and significantly improves the speed and performance of arithmetic operations in real-time systems.

3. A multiplication unit implementing Booth's algorithm gives incorrect results for certain negative operands. Debug the step-by-step execution of the algorithm, focusing on bit-pair checking, arithmetic shifts, and sign extension. Identify the possible source of error and suggest corrections to ensure accurate signed multiplication.

Debugging Booth's Multiplication for Negative Operands

Problem

A multiplication unit implementing Booth's algorithm produces incorrect results when multiplying certain negative numbers.

Analysis

Consider the multiplication:

$$-5 \times 3$$

Using 4-bit 2's complement representation:

- **Multiplicand (M) = 1011 (-5)**
- **Multiplier (Q) = 0011 (+3)**

Initialize:

- $A = 0000$
- $Q = 0011$
- $Q_{-1} = 0$

Booth's algorithm checks the pair (Q_0, Q_{-1})

Bit Pair	Operation
01	$A = A + M$
10	$A = A - M$
00 or 11	No operation

After each operation, an arithmetic right shift must be performed on A, Q, and Q_{-1} together.

Error / Cause

- a **logical right shift** is used instead of an arithmetic right shift,
- the **sign bit is not preserved** during shifting,
- or the multiplicand is **not properly sign-extended**.

f the sign bit is lost, the partial products become incorrect, especially for negative operands.

Corrected / Optimized Solution

The algorithm must:

- use **arithmetic right shift**
- preserve the **MSB sign of** all partial products,
- and perform **proper sign extension** whenever the register width is increased.

For the example:

$$-5 \times 3 = -15$$

Binary result (8-bit 2's complement): 11110001

This is the correct product.

Testing / Validation

Input:

- M = 1011 (-5)
- Q = 0011 (+3)

Expected Output: 11110001 (-15)

Observed After Correction: 11110001 (-15)

Result

The error is caused by incorrect handling of arithmetic shifts and sign extension in Booth's algorithm. By preserving the sign bit during every shift and correctly applying the add/subtract rules for each bit pair, the multiplication unit produces accurate signed multiplication results for both positive and negative operands.

4. An embedded system using the restoring division algorithm experiences inefficiency due to repeated restoration steps, leading to increased execution time. Debug the algorithm's workflow and identify where unnecessary operations occur. Propose an optimized approach using the non-restoring division method, explaining how it improves performance.

Optimizing Restoring Division

Problem

An embedded system using the restoring division algorithm shows poor performance because repeated restoration steps increase the execution time.

Analysis

In restoring division, each iteration performs the following operations:

- Shift the remainder and quotient left.

- Subtract the divisor from the remainder.
- If the remainder becomes negative, restore the previous remainder by adding the divisor again.
- Set the quotient bit accordingly.

Many iterations therefore execute:

Subtract → Negative → Restore

This means two arithmetic operations are performed even when the subtraction is unsuccessful.

Error / Cause

The inefficiency is caused by unnecessary restoration of the remainder after every unsuccessful subtraction. These repeated add-back operations increase the number of cycles and reduce overall performance.

Corrected / Optimized Solution

Use the Non-Restoring Division algorithm.

Rules:

- If the remainder is **positive**, subtract the divisor.
- If the remainder is **negative**, **add the divisor** in the next iteration.
- Do not restore immediately.

Example:

$$13 \div 3$$

The algorithm completes without repeatedly restoring the remainder.

Final result:

- Quotient = 4
- Remainder = 1

Testing / Validation

Input: Dividend = 13

Divisor = 3

Restoring Division: More arithmetic operations

Non-Restoring Division: Fewer arithmetic operations

Output:

Quotient = 4 Remainder = 1

Result

By replacing restoring division with non-restoring division, unnecessary restoration operations are eliminated. This reduces the number of arithmetic operations, decreases execution time, and improves the efficiency and speed of the embedded system.

5. A scientific application using IEEE 754 single precision floating-point representation produces inaccurate results when adding very small and very large numbers. Debug the issue by analyzing exponent alignment, normalization, and rounding errors. Suggest optimization techniques, such as using double precision or algorithmic adjustments, to improve accuracy.

Debugging IEEE 754 Floating-Point Inaccuracy

Problem

A scientific application using IEEE 754 single-precision floating-point representation produces inaccurate results when adding a very small number to a very large number.

Analysis

Consider the operation:

$$1.0 \times 10^{20} + 1.0$$

In IEEE 754 single precision:

- 1 sign bit
- 8 exponent bits
- 23 fraction (mantissa) bits

Before addition, the exponents must be aligned. The smaller number (1.0) is shifted right many times to match the exponent of 1.0×10^{20} .

After repeated shifts, its significant bits are lost due to limited mantissa precision.

Error / Cause

The inaccuracy occurs because of:

- Exponent alignment causing the smaller operand to shift right.
- Loss of significant bits during shifting.
- Normalization after addition.
- Rounding errors due to the 23-bit mantissa limit.

As a result, the small value may have no effect on the final result.

Corrected / Optimized Solution

Use double-precision (64-bit) floating point, which provides:

- 11 exponent bits
- 52 fraction bits

This preserves much more precision.

For repeated additions of small values, **use Kahan Summation**, which compensates for lost low-order bits and reduces accumulated rounding error.

Testing / Validation

Input:

- Large number = 1.0×10^{20}
- Small number = 1.0

Single Precision Output:

- 1.0×10^{20}

Double Precision Output:

- $1.00000000000000000001 \times 10^{20}$ (more accurate)

Validation:

- Expected: Large value plus small contribution
- Observed (single precision): Small contribution lost
- Observed (double precision): Small contribution preserved

Result

In single precision, the result may be stored as:

$$1.0 \times 10^{20}$$

while double precision retains a much more accurate value. The error is caused by loss of significance during exponent alignment and rounding, and it can be reduced by using higher precision and compensated summation techniques.

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

MARKS ALLOCATION

	Identification of Errors / Bugs (20%)	Debugging Approach & Analysis (20%)	Correctness of Debugged Solution (25%)	Optimization Techniques Applied (20%)	Testing and Documentation (15%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Identification of Errors / Bugs	20%	Accurately identifies all errors and issues with complete understanding of the problem	Identifies most errors with minor omissions	Identifies limited errors; misses key issues	Unable to identify errors or misunderstands the problem
Debugging Approach & Analysis	20%	Uses effective debugging techniques with clear analysis and root cause identification	Applies suitable debugging methods with minor gaps	Uses basic debugging methods with limited analysis	No systematic debugging approach followed
Correctness of Debugged Solution	25%	Provides completely corrected solution with accurate functionality and expected output	Provides working solution with minor errors	Partially correct solution with limited functionality	Incorrect solution or fails to resolve errors
Optimization Techniques Applied	20%	Applies efficient optimization methods to improve performance, memory usage and quality	Performs optimization with noticeable improvements	Limited optimization with minimal improvements	No optimization applied or inefficient implementation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Testing and Documentation	15%	Performs complete testing with proper validation and clear documentation	Provides adequate testing and documentation with minor issues	Limited testing and incomplete documentation	No proper testing or documentation provided